

HelixTerminator

HelixTerminator

La plateforme terminale à confiance zéro pour les équipes — chaque session SSH sécurisée, partagée et assistée par AI.

Résumé

HelixTerminator est une plateforme d'entreprise pour terminal et gestion de sessions SSH, conçue comme un système de microservices Go avec des clients Flutter multiplateformes. Elle gère, enregistre et sécurise les sessions à distance selon un modèle de confiance zéro, intègre la collaboration en temps réel et superpose une assistance AI au terminal.

Description courte

HelixTerminator est une plateforme terminale d'entreprise à confiance zéro et de gestion de sessions SSH : un backend en microservices Go associé à des clients Flutter couvrant six plateformes. Elle gère les hôtes, établit les connexions, enregistre les sessions, permet la collaboration en temps réel et ajoute une assistance AI pour l'aide aux commandes, l'explication des sorties et la réponse aux incidents.

Description détaillée

HelixTerminator est une plateforme terminale et d'accès à distance de niveau entreprise, structurée en deux modules — une Plateforme Terminal et un Courtier de Connexions — déployés via un catalogue de microservices Go avec un client Flutter unique prenant en charge six plateformes. Son ambition est de remplacer entièrement les outils SSH ad hoc : plus de clients par machine, de prolifération de clés privées ou de lacunes d'audit, mais un

système unique, gouverné, auditable et collaboratif qui considère l'accès à distance comme une infrastructure plutôt qu'une habitude personnelle.

Le backend gère l'intégralité du cycle de vie de l'accès à distance de bout en bout. Les hôtes et groupes sont administrés via des chaînes de bastions/serveurs relais ; un proxy SSH gère l'authentification par mot de passe, clé publique et certificat ; un proxy d'E/S terminal diffuse la session via WebSocket ; SFTP prend en charge les transferts résumables ; et des fonctionnalités telles que le transfert de ports, la gestion d'extraits et d'espaces de travail, ainsi que l'enregistrement de sessions sont assemblées en lectures asciinema signées, reproductibles et fiables. La sécurité repose sur une approche de confiance zéro par conception, et non en tant qu'ajout ultérieur : un coffre-fort assure le stockage secret en mode zéro connaissance, un service PKI émet des certificats SSH à courte durée de vie pour éviter que des identifiants permanents ne traînent et ne soient volés, des trousseaux de clés matériels (Secure Enclave / Android Keystore / DPAPI / HSM) maintiennent les clés hors du disque, FIDO2/WebAuthn et OIDC/SAML sécurisent l'authentification, et un journal d'audit en appendice, chaîné par Merkle, produit des preuves infalsifiables pour les normes SOC 2 et ISO 27001. Par ailleurs, la collaboration en temps réel permet à plusieurs opérateurs de partager une même session en direct, avec des rôles d'observateur, de copilote ou de propriétaire, synchronisés par une synchronisation de tampons CRDT.

Un service AI s'intègre au terminal lui-même, offrant l'autocomplétion des commandes, des explications en langage clair des sorties, la détection d'anomalies, la génération de procédures et une assistance pratique en cas d'incident — transformant le terminal d'un simple canal en un assistant aux moments critiques. L'ensemble de la plateforme est natif pour les conteneurs — Kubernetes, Helm, Terraform, et une pile complète d'observabilité incluant OpenTelemetry, Grafana, Jaeger et Loki — et s'intègre à l'écosystème Helix via un pont HelixTrack et un agent local HelixLLM. Le tout fonctionne sous le contrôle du Constitution Helix, avec des portes de vérification d'héritage anti-usurpation.

Contenu

Pourquoi nous l'avons conçu

Les équipes gèrent des infrastructures distantes via des clients SSH dispersés, sans piste d'audit partagée, sans gestion cohérente des secrets et sans moyen de collaborer en direct sur

un incident. HelixTerminator a été créé pour transformer l'accès distant en une plateforme gouvernée, *zero-trust* et conçue pour les équipes, plutôt qu'en un outil par poste de travail.

Pourquoi c'est révolutionnaire

Il condense une liste d'achats entière en une seule plateforme. Le client SSH, le coffre-fort de secrets, la couche bastion/PKI, l'enregistrement des sessions, l'audit de conformité et la collaboration en temps réel sont autant d'éléments que les équipes achètent, assemblent et réconcilient habituellement séparément – chacun présentant ses propres lacunes aux points de jonction. HelixTerminator les intègre en un système unifié et gouverné, puis va plus loin que ces outils pris isolément : il superpose une couche AI directement sur le terminal pour expliquer les sorties inconnues et rédiger des procédures *pendant qu'un incident est en cours*. Une capacité autrefois irréalisable devient réalité : une session d'accès distant à la fois sécurisée en *zero-trust*, enregistrée de manière infalsifiable, partagée en direct entre opérateurs et assistée par AI – le tout depuis une seule interface.

Ce qui est innovant

- **Architecture à double module** (Plateforme Terminal + Courtier de Connexion) coordonnée via un registre de services, permettant à la plateforme et à la couche de courtage d'évoluer et de s'adapter indépendamment.
- **Sécurité *zero-trust* de bout en bout** : certificats SSH éphémères émis par une PKI, coffre-fort à connaissance nulle, trousseaux de clés matériels et journal d'audit chaîné par Merkle – aucun identifiant permanent, aucune piste invérifiable.
- **Collaboration en temps réel sur les sessions** grâce à une synchronisation de tampon basée sur les CRDT et des rôles explicites (observateur / copilote / propriétaire), permettant à plusieurs opérateurs de travailler sur un même terminal sans se marcher sur les pieds.
- **Assistance AI intégrée aux opérations** directement dans le terminal en direct : autocomplétion, explication des sorties, détection d'anomalies et assistance aux procédures/incidents là où l'opérateur en a besoin.
- **Client Flutter multiplateforme** couvrant six environnements à partir d'une seule base de code, garantissant une expérience cohérente sur les versions desktop, mobile et web.

Principaux défis techniques et solutions apportées

- **Sécuriser l'accès distant sans aucun identifiant permanent à voler** – les clés à longue durée de vie sont un vecteur de compromission classique. Solution : un service PKI émettant des certificats SSH éphémères à la demande, un coffre-fort à connaissance nulle stockant des secrets que le serveur lui-même ne peut pas lire, et un stockage matériel des clés (Secure Enclave / Android Keystore / DPAPI / HSM) pour éviter que les données privées ne restent exposées sur le disque.
- **Permettre à plusieurs opérateurs de piloter une même session sans corrompre le tampon** – les modifications concurrentes sur un terminal partagé posent un problème de cohérence complexe. Solution : une synchronisation de tampon basée sur les CRDT, préférée à la transformation opérationnelle (conformément à l'ADR-006), car les CRDT convergent sans arbitre central.
- **Rendre les preuves de conformité impossibles à altérer discrètement** – un journal d'audit modifiable ne prouve rien. Solution : un journal en écriture seule, chaîné par Merkle, où toute tentative de falsification brise la chaîne de hachage, produisant des preuves exportables pour les normes SOC 2 / ISO 27001 / FedRAMP.
- **Offrir une expérience utilisateur cohérente sur desktop, mobile et web sans maintenir trois bases de code** – solution : un client Flutter/Dart unique suivant le modèle BLoC, Flutter étant préféré à Electron (conformément à l'ADR-001) pour cibler six plateformes à partir d'une seule source de vérité.

Contenu

Pile technologique

- **Microservices Go** — la flotte backend (proxy SSH, terminal, coffre-fort, PKI, audit, et autres) ; choisis pour leur modèle de concurrence et leur empreinte d'exécution réduite, idéaux pour des services gérant de nombreuses sessions de streaming longues simultanées (ADR-002 : Go plutôt que Rust/Node).
- **Flutter / Dart (BLoC)** — un seul code client pour six plateformes, avec BLoC assurant une gestion prévisible de l'état ; Flutter préféré à Electron (ADR-001) pour éviter de maintenir des interfaces natives et web distinctes.
- **PostgreSQL** — la base de données principale, choisie plutôt que CockroachDB (ADR-004) pour son cœur transactionnel mature et bien maîtrisé.

- **Kafka + RabbitMQ** — la couche de messagerie et de streaming transportant les segments de session et les événements (ADR-003), combinant un journal durable avec une file d'attente flexible.
- **Redis** — stocke les tampons de défilement des terminaux et l'état des sessions actives lorsque la faible latence prime sur la durabilité.
- **SPIFFE/SPIRE + mTLS** — émet des identités cryptographiques pour les charges de travail (ADR-005), permettant une authentification mutuelle du trafic inter-services et étendant le principe de *zero trust* au sein du maillage, et non seulement à sa périphérie.
- **Ed25519 (EdDSA)** — signe les JWT et les enregistrements de session (ADR-009), offrant des signatures modernes et rapides qui rendent les sessions enregistrées vérifiables.
- **Kubernetes + Helm + Terraform** — déploiement natif en conteneurs avec une infrastructure reproductible et versionnée (ADR-007/008).
- **OpenTelemetry, Grafana, Jaeger, Loki** — la pile d'observabilité pour le traçage, les métriques, les tableaux de bord et les journaux ; **Falco, Trivy, Cosign, Sealed Secrets** — détection des menaces en temps réel, analyse des images, signature des artefacts et livraison chiffrée des secrets tout au long de la chaîne d'approvisionnement.

Statut et notes de transparence

- **Statut : bêta.** Une base de code substantielle et en développement actif (créée le 4 juillet 2026). Les chiffres de spécifications numériques du dossier de recherche MVP du projet (nombre de points de terminaison, de tables et de services) sont des objectifs de conception/spécification issus de docs/research/mvp/, et non confirmés comme pleinement implémentés. Ils sont donc présentés ci-dessus comme une portée architecturale plutôt que comme des métriques livrées. Les affirmations concernant la latence, les SLO et le caractère « prêt pour la production » n'ont pas été vérifiées de manière indépendante.
- **Licence : Apache-2.0** (conformément à GitHub API).

Niveau de priorité : Helix-primaire.